

Methodology and Results: Spatiotemporal Predictive Model for Traffic Incidents

Code Blooded

June 21, 2026

1 Introduction

This document outlines the methodology employed to build a spatiotemporal predictive model using a small traffic incident dataset. To prevent overfitting given the dataset's limited size, deep learning approaches were strictly avoided. Instead, a robust pipeline leveraging tree-based models and signal processing techniques was implemented to forecast incident volumes across different geographic zones.

2 Data Preprocessing and Signal Processing

The raw traffic incident dataset was processed by aggregating incident occurrences into 1-hour windows per zone. Because the data was noisy, a Savitzky-Golay filter was applied to smooth the aggregated incident frequencies. Additionally, cyclical time features were engineered using sine and cosine transformations of the hour-of-day and day-of-week, and spatial variables such as zone and junction were label-encoded.

2.1 Savitzky-Golay Filter

The Savitzky-Golay filter is a digital filter that smooths data by fitting successive sub-sets of adjacent data points with a low-degree polynomial using linear least squares. For a given point i in a sliding window of size $2m + 1$, the smoothed value y_i^* is computed as:

$$y_i^* = \sum_{j=-m}^m C_j y_{i+j} \quad (1)$$

where y is the raw signal (incident count), m is the half-window size, and C_j are the convolution coefficients derived from the least squares fit of a polynomial (of order 3 in our methodology). This technique preserves the features of the data such as local maxima and minima, which are critical for capturing incident spikes, while effectively reducing high-frequency noise.

3 Modeling Framework

The predictive modeling was carried out using the XGBoost (Extreme Gradient Boosting) algorithm. The model leverages gradient boosting of decision trees with explicit regularization to control overfitting.

3.1 XGBoost Objective Function

At each step t , the XGBoost algorithm minimizes the following objective function (which includes both a loss term and a regularization term):

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t) \quad (2)$$

where l is a differentiable convex loss function measuring the difference between the target y_i and the prediction $\hat{y}_i$. For this regression task, we utilized the squared error loss. The regularization term $\Omega(f)$ penalizes the complexity of the model:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2 + \alpha \|\mathbf{w}\|_1 \quad (3)$$

Here, T is the number of leaves in the tree, $\mathbf{w}$ contains the leaf weights, and γ, λ, α are regularization parameters. Our hyperparameter tuning prioritized optimizing λ (L2 regularization) and α (L1 regularization) alongside the maximum tree depth to explicitly mitigate overfitting.

4 Validation Strategy and Results

Due to the temporal nature of the data, a `TimeSeriesSplit` cross-validation strategy was applied, ensuring that the model is always trained on past data to predict future incidents.

4.1 Evaluation Metrics

The performance of the model is evaluated using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE), defined as:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad \text{and} \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4)$$

4.2 Results

After rigorous hyperparameter tuning focused on regularization, the best model achieved the following metrics on the evaluation splits:

- **Cross-Validation RMSE:** 0.0664
- **Cross-Validation MAE:** 0.0191
- **Hold-out Test RMSE:** 0.5643
- **Hold-out Test MAE:** 0.0876

These results indicate that the combination of signal processing and explicitly regularized tree-based models provides a robust forecasting mechanism for noisy, small-sample spatiotemporal traffic data without overfitting.

5 Prescriptive Logistics Layer

To bridge the gap between predictive outputs and actionable operations, we built a prescriptive logistics layer that directly interfaces with the XGBoost predictions. This layer provides optimal resource allocation and dynamic routing recommendations.

5.1 Resource Allocation via Integer Linear Programming (ILP)

To efficiently deploy police personnel and barricades to high-risk zones, we formulated an Integer Linear Programming (ILP) problem. Let Z denote the set of geographic zones and w_i the predicted incident risk for zone $i \in Z$. Let P_i and B_i represent the integer number of personnel and barricades allocated to zone i , respectively.

The objective is to maximize the resource coverage, heavily weighted toward zones with high predicted risk:

$$\max \sum_{i \in Z} w_i (P_i + B_i) \quad (5)$$

Subject to the total availability constraints and arbitrary zone capacities to prevent resource hoarding:

$$\sum_{i \in Z} P_i \leq P_{max} \quad (6)$$

$$\sum_{i \in Z} B_i \leq B_{max} \quad (7)$$

$$P_i, B_i \in \{0, 1, \dots, 10\} \quad \forall i \in Z \quad (8)$$

This guarantees that operational capacities are strictly respected while mathematically prioritizing high-risk hotspots.

5.2 Dynamic Diversion Routing

When the XGBoost model flags a specific junction J_{risk} as highly prone to imminent incidents, the system recalculates routing for traffic. We represent the road network as a graph $G = (V, E)$, where edges $(u, v) \in E$ have baseline travel time weights c_{uv} .

Upon flagging a high-risk junction, a penalty Δ is dynamically applied to all edges connected to it. The updated edge weights c'_{uv} are given by:

$$c'_{uv} = \begin{cases} c_{uv} + \Delta & \text{if } u = J_{risk} \text{ or } v = J_{risk} \\ c_{uv} & \text{otherwise} \end{cases} \quad (9)$$

Dijkstra's shortest-path algorithm is then executed on the modified graph G' to discover alternative routes that autonomously circumvent the predicted bottleneck.